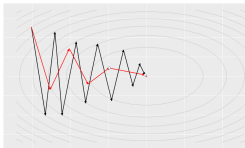


# Advanced Artificial Intelligence and Machine Learning

## Optimizers

---



### Learning goals

- Momentum-based and Nesterov-accelerated SGD
- Learning rate schedules and adaptive learning-rate methods (Adagrad, RMSProp, Adam)
- Batch normalization and its effect on the optimization landscape
- Hyperparameter tuning: the search problem, grid/random search, Bayesian optimization, multi-fidelity methods, and the untouched-test-set principle
- Neural Architecture Search: what it automates and why, illustrated with one concrete example (DARTS)

## SGD Recap

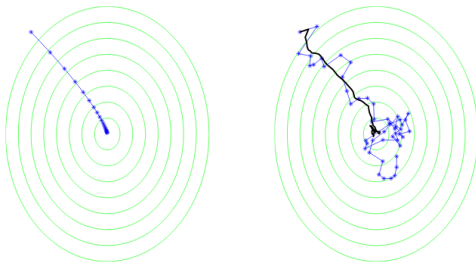
# SGD RECAP

Full-batch gradient descent for ERM:

$$\boldsymbol{\theta}^{[t+1]} = \boldsymbol{\theta}^{[t]} - \alpha \cdot \frac{1}{n} \cdot \sum_{i=1}^n \nabla_{\boldsymbol{\theta}} L \left( y^{(i)}, f(\mathbf{x}^{(i)} \mid \boldsymbol{\theta}^{[t]}) \right)$$

- Using the **entire** training set for every update (**batch/deterministic** training) is computationally costly, or impossible if the data doesn't fit in memory.
- **Idea:** sum over a small, randomly sampled **minibatch** instead — if sampled uniformly, this is, in expectation, the true batch gradient: a **noisy** version of batch GD.

## SGD RECAP / 2



**Figure:** SGD on  $1.25(x_1 + 6)^2 + (x_2 - 8)^2$ . Left: GD; right: SGD. The black line is a smoothed  $\theta$  trajectory (Shalev-Shwartz & Ben-David, 2022).

# SGD IN PRACTICE

- A full SGD pass over the data is one **epoch**; typical minibatch sizes are 50–1000.
- SGD is the most-used optimizer in deep learning — but usually needs extra tricks to be efficient. That's what the rest of this session is about: momentum, adaptive learning rates, and batch normalization.
- Small-batch SGD is unbiased but has **high variance**, so its learning rate  $\alpha$  is typically smaller than in full-batch mode.
- With a slowly decreasing learning rate, SGD converges to a local minimum — and often generalizes **better** than full-batch GD, acting as an indirect regularizer.

# REFERENCES



Shalev-Shwartz, S., & Ben-David, S. (2022). *Understanding machine learning: From theory to algorithms*. Cambridge University Press.

# Momentum

---

# MOMENTUM

- While SGD remains a popular optimization strategy, learning with it can sometimes be slow.
- Momentum is designed to accelerate learning, especially when facing high curvature, small but consistent or noisy gradients.
- Momentum accumulates an exponentially decaying moving average of past gradients:

$$\begin{aligned}\boldsymbol{\nu} &\leftarrow \varphi \boldsymbol{\nu} - \underbrace{\alpha \nabla_{\boldsymbol{\theta}} \left[ \frac{1}{m} \sum_i L(y^{(i)}, f(x^{(i)}, \boldsymbol{\theta})) \right]}_{g_{\boldsymbol{\theta}}} \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \boldsymbol{\nu}\end{aligned}$$

- We introduce a new hyperparameter  $\varphi \in [0, 1)$ , determining how quickly the contribution of previous gradients decay.

## MOMENTUM / 2

- $\nu$  is called “velocity” and derives from a physical analogy describing how particles move through a parameter space (Newton’s law of motion).

# MOMENTUM

- So far the step size was simply the gradient  $g$  multiplied by the learning rate  $\alpha$ .
- Now, the step size depends on how **large** and how **aligned** a sequence of gradients is. The step size grows when many successive gradients point in the same direction.
- Common values for  $\varphi$  are 0.5, 0.9 and even 0.99.
- Generally, the larger  $\varphi$  is relative to the learning rate  $\alpha$ , the more previous gradients affect the current direction.
- A very good website with an in-depth analysis of momentum:  
<https://distill.pub/2017/momentum/>

# MOMENTUM: EXAMPLE

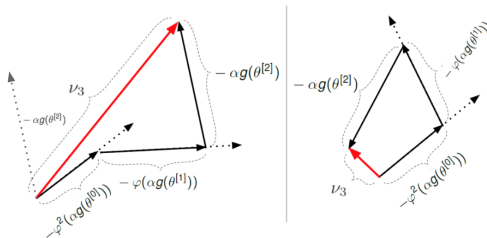
## MOMENTUM: EXAMPLE / 2

$$\begin{aligned}\nu_1 &\leftarrow \varphi \nu_0 - \alpha g(\theta^{[0]}) \\ \theta^{[1]} &\leftarrow \theta^{[0]} + \varphi \nu_0 - \alpha g(\theta^{[0]}) \\ \nu_2 &\leftarrow \varphi \nu_1 - \alpha g(\theta^{[1]}) \\ &= \varphi(\varphi \nu_0 - \alpha g(\theta^{[0]})) - \alpha g(\theta^{[1]}) \\ \theta^{[2]} &\leftarrow \theta^{[1]} + \varphi(\varphi \nu_0 - \alpha g(\theta^{[0]})) - \alpha g(\theta^{[1]}) \\ \nu_3 &\leftarrow \varphi \nu_2 - \alpha g(\theta^{[2]}) \\ &= \varphi(\varphi(\varphi \nu_0 - \alpha g(\theta^{[0]})) - \alpha g(\theta^{[1]})) - \alpha g(\theta^{[2]}) \\ \theta^{[3]} &\leftarrow \theta^{[2]} + \varphi(\varphi(\varphi \nu_0 - \alpha g(\theta^{[0]})) - \alpha g(\theta^{[1]})) - \alpha g(\theta^{[2]}) \\ &= \theta^{[2]} + \varphi^3 \nu_0 - \varphi^2 \alpha g(\theta^{[0]}) - \varphi \alpha g(\theta^{[1]}) - \alpha g(\theta^{[2]}) \\ &= \theta^{[2]} - \alpha(\varphi^2 g(\theta^{[0]}) + \varphi^1 g(\theta^{[1]}) + \varphi^0 g(\theta^{[2]})) + \varphi^3 \nu_0 \\ \theta^{[t+1]} &= \theta^{[t]} - \alpha \sum_{j=0}^t \varphi^j g(\theta^{[t-j]}) + \varphi^{t+1} \nu_0\end{aligned}$$

# MOMENTUM: ILLUSTRATION

The vector  $\nu_3$  (for  $\nu_0 = 0$ ):

$$\begin{aligned}\nu_3 &= \varphi(\varphi(\varphi\nu_0 - \alpha g(\theta^{[0]})) - \alpha g(\theta^{[1]})) - \alpha g(\theta^{[2]}) \\ &= -\varphi^2(\alpha g(\theta^{[0]})) - \varphi(\alpha g(\theta^{[1]})) - \alpha g(\theta^{[2]})\end{aligned}$$



**Figure:** If consecutive (negative) gradients point mostly in the same direction, the velocity "builds up". On the other hand, if consecutive (negative) gradients point in very different directions, the velocity "dies down".

# SGD WITH MOMENTUM

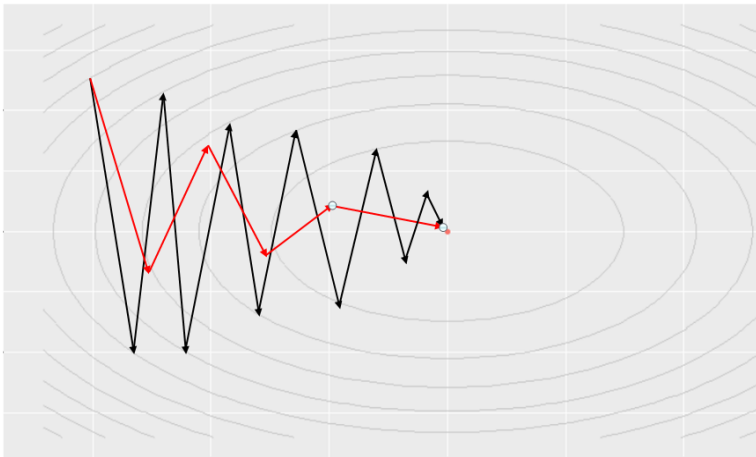
---

## Algorithm Stochastic gradient descent with momentum

---

- 1: **require** learning rate  $\alpha$  and momentum  $\varphi$
  - 2: **require** initial parameter  $\theta$  and initial velocity  $\nu$
  - 3: **while** stopping criterion not met **do**
  - 4:     Sample a minibatch of  $m$  examples from the training set  $\{\tilde{x}^{(1)}, \dots, \tilde{x}^{(m)}\}$
  - 5:     Compute gradient estimate:  $\hat{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(y^{(i)}, f(\tilde{x}^{(i)} | \theta))$
  - 6:     Compute velocity update:  $\nu \leftarrow \varphi \nu - \alpha \hat{g}$
  - 7:     Apply update:  $\theta \leftarrow \theta + \nu$
  - 8: **end while**
-

## SGD WITH MOMENTUM / 2

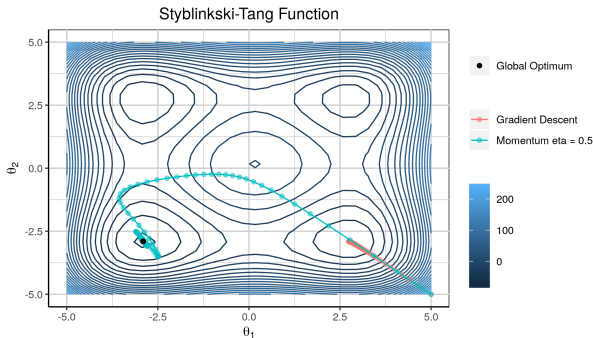


## SGD WITH MOMENTUM / 3

**Figure:** The contour lines show a quadratic loss function with a poorly conditioned Hessian matrix. The two curves show how standard gradient descent (black) and momentum (red) learn when dealing with ravines. Momentum reduces the oscillation and accelerates the convergence.

# SGD WITH AND WITHOUT MOMENTUM

The following plot was created by our Shiny App. On the upper left you can explore different predefined examples. [▶ Click here](#)



# SGD WITH AND WITHOUT MOMENTUM

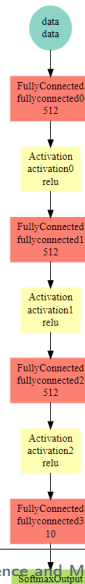
/ 2

---

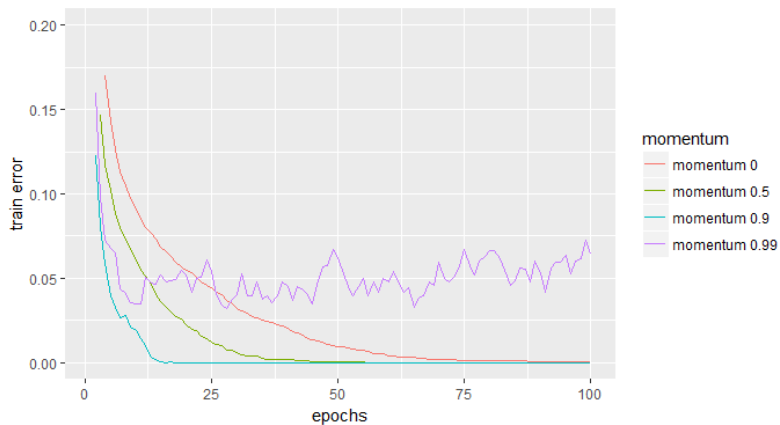
**Figure:** Comparison of SGD with and without momentum on the Styblinski-Tang function. The black dot on the bottom left is the global optimum. We can see that SGD without momentum (red line/points) cannot escape the local minimum, while SGD with momentum (blue line/dots) is able to escape the local minimum and finds the global minimum.

# MOMENTUM IN PRACTICE

- Lets try out different values of momentum (with SGD) on the MNIST data.
- We apply the same architecture we have used a dozen of times already (note that we used  $\varphi = 0.9$  in all computations so far, i.e. in chapter 1 and 2)!



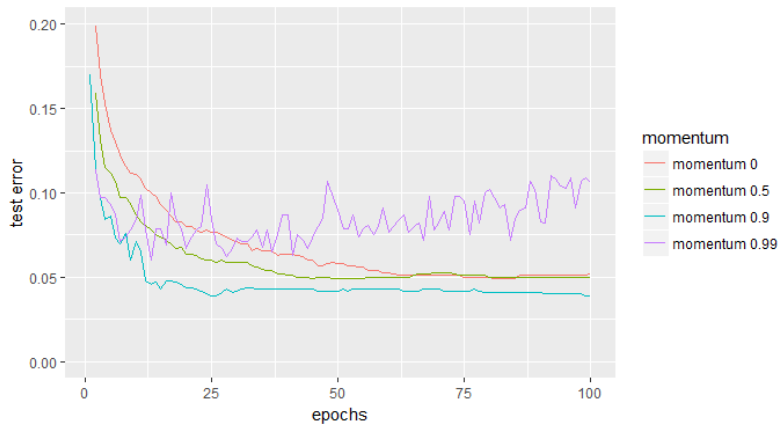
# MOMENTUM IN PRACTICE



## MOMENTUM IN PRACTICE / 2

The higher momentum, the faster SGD learns the weights on the training data, but if momentum is too large, the training and test error fluctuates.

## MOMENTUM IN PRACTICE / 3



## MOMENTUM IN PRACTICE / 4

The higher momentum, the faster SGD learns the weights on the training data, but if momentum is too large, the training and test error fluctuates.

# NESTEROV MOMENTUM

- Momentum aims to solve poor conditioning of the Hessian but also variance in the stochastic gradient.
- Nesterov momentum modifies the algorithm such that the gradient is evaluated after the current velocity is applied:

$$\begin{aligned}\boldsymbol{\nu} &\leftarrow \varphi \boldsymbol{\nu} - \alpha \nabla_{\boldsymbol{\theta}} \left[ \frac{1}{m} \sum_i L(y^{(i)}, f(x^{(i)}, \boldsymbol{\theta} + \varphi \boldsymbol{\nu})) \right] \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \boldsymbol{\nu}\end{aligned}$$

- We can interpret Nesterov momentum as an attempt to add a correction factor to the basic method.
- The method is also called Nesterov accelerated gradient (NAG).

# SGD WITH NESTEROV MOMENTUM

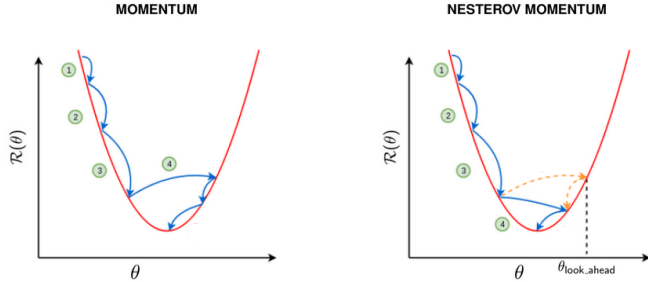
---

**Algorithm** Stochastic gradient descent with Nesterov momentum

---

- 1: **require** learning rate  $\alpha$  and momentum  $\varphi$
  - 2: **require** initial parameter  $\theta$  and initial velocity  $\nu$
  - 3: **while** stopping criterion not met **do**
  - 4:     Sample a minibatch of  $m$  examples from the training set  $\{\tilde{x}^{(1)}, \dots, \tilde{x}^{(m)}\}$
  - 5:     Apply interim update:  $\tilde{\theta} \leftarrow \theta + \varphi \nu$
  - 6:     Compute gradient estimate:  $\hat{g} \leftarrow \frac{1}{m} \nabla_{\tilde{\theta}} \sum_i L(y^{(i)}, f(x^{(i)}, \tilde{\theta}))$
  - 7:     Compute velocity update:  $\nu \leftarrow \varphi \nu - \alpha \hat{g}$
  - 8:     Apply update:  $\theta \leftarrow \theta + \nu$
  - 9: **end while**
-

# MOMENTUM VS. NESTEROV MOMENTUM



Credits: Chandra (2015)

# MOMENTUM VS. NESTEROV

## MOMENTUM / 2

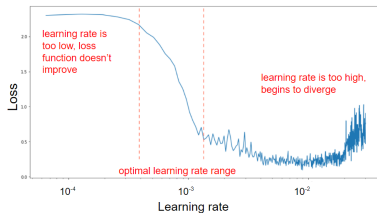
**Figure:** Comparison GD with momentum (left) and GD with Nesterov momentum (right) for one parameter  $\theta$ . The first three updates of  $\theta$  are very similar in both cases and the updates become larger due to momentum (accumulation of previous negative gradients). Update 4 is different. In case of momentum, the update overshoots as it makes an even bigger step due to the gradient history. In contrast, Nesterov momentum first evaluates a "look-ahead" point  $\theta_{\text{look\_ahead}}$ , detects that it overshoots, and slightly reduces the overall magnitude of the fourth update. Thus, Nesterov momentum reduces overshooting and leads to smaller oscillations than momentum.

# Learning Rates

---

# LEARNING RATE

- The learning rate is a very important hyperparameter.
- To systematically find a good learning rate, we can start at a very low learning rate and gradually increase it (linearly or exponentially) after each mini-batch.
- We can then plot the learning rate and the training loss for each batch.
- A good learning rate is one that results in a steep decline in the loss.



Credit: jeremyjordan

# LEARNING RATE SCHEDULE

- We would like to force convergence until reaching a local minimum.
- Applying SGD, we have to decrease the learning rate over time, thus  $\alpha^{[t]}$  (learning rate at training iteration  $t$ ).
  - The estimator  $\hat{g}$  is computed based on small batches.
  - Random sampling  $m$  training samples introduces noise, that does not vanish even if we find a minimum.
- In practice, a common strategy is to decay the learning rate linearly over time until iteration  $\tau$ :

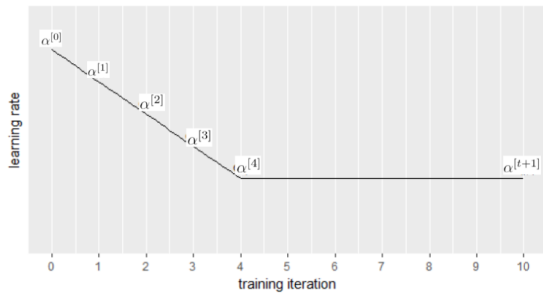
$$\alpha^{[t]} = \begin{cases} \left(1 - \frac{t}{\tau}\right) \alpha^{[0]} + \frac{t}{\tau} \alpha^{[\tau]} = t \left(-\frac{\alpha^{[0]} + \alpha^{[\tau]}}{\tau}\right) + \alpha^{[0]} & \text{for } t \leq \tau \\ \alpha^{[\tau]} & \text{for } t > \tau \end{cases}$$

# LEARNING RATE SCHEDULE / 2

Example for  $\tau = 4$ :

| iteration t | $t/\tau$ | $\alpha^{[t]}$                                                                                                          |
|-------------|----------|-------------------------------------------------------------------------------------------------------------------------|
| 1           | 0.25     | $(1 - \frac{1}{4}) \alpha^{[0]} + \frac{1}{4} \alpha^{[\tau]} = \frac{3}{4} \alpha^{[0]} + \frac{1}{4} \alpha^{[\tau]}$ |
| 2           | 0.5      | $\frac{2}{4} \alpha^{[0]} + \frac{2}{4} \alpha^{[\tau]}$                                                                |
| 3           | 0.75     | $\frac{1}{4} \alpha^{[0]} + \frac{3}{4} \alpha^{[\tau]}$                                                                |
| 4           | 1        | $0 + \alpha^{[\tau]}$                                                                                                   |
| ...         |          | $\alpha^{[\tau]}$                                                                                                       |
| $t + 1$     |          | $\alpha^{[\tau]}$                                                                                                       |

# LEARNING RATE SCHEDULE / 3

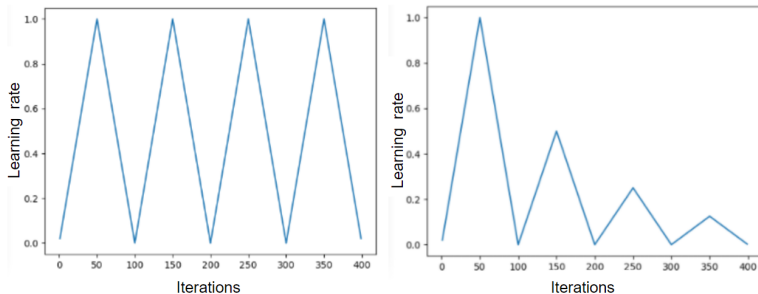


# CYCLICAL LEARNING RATES

- Another option is to have a learning rate that periodically varies according to some cyclic function.
- Therefore, if training does not improve the loss anymore (possibly due to saddle points), increasing the learning rate makes it possible to rapidly traverse such regions.
- Recall, saddle points are far more likely than local minima in deep nets.
- Each cycle has a fixed length in terms of the number of iterations.

# CYCLICAL LEARNING RATES

- One such cyclical function is the "triangular" function.

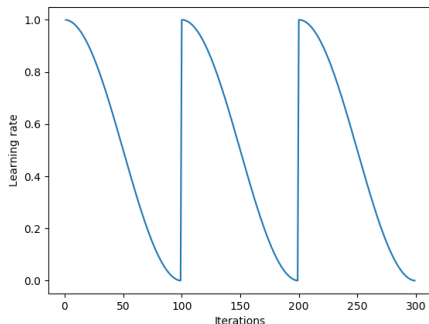


Credit: Hafidz Zulkifli

- In the right image, the range is cut in half after each cycle.

# CYCLICAL LEARNING RATES

- Yet another option is to abruptly "restart" the learning rate after a fixed number of iterations.
- Loshchilov et al. (2016) proposed "cosine annealing" (between restarts).



Credit: Hafidz Zulkifli

# Algorithms with Adaptive Learning Rates

---

# ADAPTIVE LEARNING RATES

- The learning rate is reliably one of the hyperparameters that is the most difficult to set because it has a significant impact on the models performance.
- Naturally, it might make sense to use a different learning rate for each parameter, and automatically adapt them throughout the training process.

# ADAGRAD

- Adagrad adapts the learning rate to the parameters.
- In fact, Adagrad scales learning rates inversely proportional to the square root of the sum of the past squared derivatives.
  - Parameters with large partial derivatives of the loss obtain a rapid decrease in their learning rate.
  - Parameters with small partial derivatives on the other hand obtain a relatively small decrease in their learning rate.
- For that reason, Adagrad might be well suited when dealing with sparse data.
- Goodfellow et al. (2016) say that the accumulation of squared gradients can result in a premature and overly decrease in the learning rate.

---

## Algorithm Adagrad

---

- 1: **require** Global learning rate  $\alpha$
  - 2: **require** Initial parameter  $\theta$
  - 3: **require** Small constant  $\beta$ , perhaps  $10^{-7}$ , for numerical stability
  - 4: **Initialize** gradient accumulation variable  $r = 0$
  - 5: **while** stopping criterion not met **do**
  - 6:     Sample a minibatch of  $m$  examples from the training set  $\{\tilde{x}^{(1)}, \dots, \tilde{x}^{(m)}\}$
  - 7:     Compute gradient estimate:  $\hat{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(y^{(i)}, f(\tilde{x}^{(i)} | \theta))$
  - 8:     Accumulate squared gradient  $r \leftarrow r + \hat{g} \odot \hat{g}$
  - 9:     Compute update:  $\nabla \theta = -\frac{\alpha}{\beta + \sqrt{r}} \odot \hat{g}$  (division and square root applied element-wise)
  - 10:    Apply update:  $\theta \leftarrow \theta + \nabla \theta$
  - 11: **end while**
- 

- “ $\odot$ ” is called Hadamard or element-wise product.
- Example:

## ADAGRAD / 3

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}, \text{ then } A \odot B = \begin{bmatrix} 1 \cdot 5 & 2 \cdot 6 \\ 3 \cdot 7 & 4 \cdot 8 \end{bmatrix}$$

# RMSPROP

- RMSprop is a modification of Adagrad.
- It's intention is to resolve Adagrad's radically diminishing learning rates.
- The gradient accumulation is replaced by an exponentially weighted moving average.
- Theoretically, that leads to performance gains in non-convex scenarios.
- Empirically, RMSProp is a very effective optimization algorithm. Particularly, it is employed routinely by deep learning practitioners.

# RMSPROP / 2

---

## Algorithm RMSProp

---

- 1: **require** Global learning rate  $\alpha$  and decay rate  $\rho \in [0, 1)$
  - 2: **require** Initial parameter  $\theta$
  - 3: **require** Small constant  $\beta$ , perhaps  $10^{-6}$ , for numerical stability
  - 4: Initialize gradient accumulation variable  $r = 0$
  - 5: **while** stopping criterion not met **do**
  - 6:   Sample a minibatch of  $m$  examples from the training set  $\{\tilde{x}^{(1)}, \dots, \tilde{x}^{(m)}\}$
  - 7:   Compute gradient estimate:  $\hat{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(y^{(i)}, f(\tilde{x}^{(i)} | \theta))$
  - 8:   Accumulate squared gradient  $r \leftarrow \rho r + (1 - \rho) \hat{g} \odot \hat{g}$
  - 9:   Compute update:  $\nabla \theta = -\frac{\alpha}{\beta + \sqrt{r}} \odot \hat{g}$
  - 10:   Apply update:  $\theta \leftarrow \theta + \nabla \theta$
  - 11: **end while**
-

# ADAM

- Adaptive Moment Estimation (Adam) is another method that computes adaptive learning rates for each parameter.
- Adam uses the first and the second moments of the gradients.
  - Adam keeps an exponentially decaying average of past gradients (first moment).
  - Like RMSProp it stores an exponentially decaying average of past squared gradients (second moment).
  - Thus, it can be seen as a combination of RMSProp and momentum.
- Basically Adam uses the combined averages of previous gradients at different moments to give it more “persuasive power” to adaptively update the parameters.

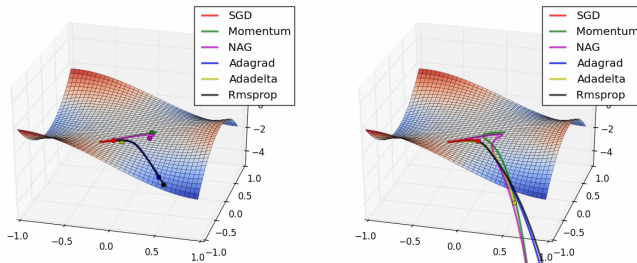
---

## Algorithm Adam

---

- 1: **require** Step size  $\alpha$  (suggested default: 0.001)
- 2: **require** Exponential decay rates for moment estimates,  $\rho_1$  and  $\rho_2$  in  $[0, 1)$  (suggested defaults: 0.9 and 0.999 respectively)
- 3: **require** Small constant  $\beta$  (suggested default  $10^{-8}$ )
- 4: **require** Initial parameters  $\theta$
- 5: Initialize time step  $t = 0$
- 6: Initialize 1st and 2nd moment variables  $s^{[0]} = 0, r^{[0]} = 0$
- 7: **while** stopping criterion not met **do**
- 8:      $t \leftarrow t + 1$
- 9:     Sample a minibatch of  $m$  examples from the training set  $\{\tilde{x}^{(1)}, \dots, \tilde{x}^{(m)}\}$
- 10:     Compute gradient estimate:  $\hat{g}^{[t]} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(y^{(i)}, f(\tilde{x}^{(i)} | \theta))$
- 11:     Update biased first moment estimate:  $s^{[t]} \leftarrow \rho_1 s^{[t-1]} + (1 - \rho_1) \hat{g}^{[t]}$
- 12:     Update biased second moment estimate:  $r^{[t]} \leftarrow \rho_2 r^{[t-1]} + (1 - \rho_2) \hat{g}^{[t]} \odot \hat{g}^{[t]}$
- 13:     Correct bias in first moment:  $\hat{s} \leftarrow \frac{s^{[t]}}{1 - \rho_1^t}$
- 14:     Correct bias in second moment:  $\hat{r} \leftarrow \frac{r^{[t]}}{1 - \rho_2^t}$
- 15:     Compute update:  $\nabla \theta = -\alpha \frac{\hat{s}}{\sqrt{\hat{r} + \beta}}$
- 16:     Apply update:  $\theta \leftarrow \theta + \nabla \theta$
- 17: **end while**

# COMPARISON OF OPTIMIZERS: ANIMATION



Credits: Dettmers (2015) and Radford

**Figure:** Excerpts from an animation to compare the behavior of momentum and other methods compared to SGD for a saddle point. Left: After a few seconds; Right: A bit later. The animation shows that all showed methods accelerate optimization compared to the standard SGD. The highest acceleration is obtained using Rmsprop followed by Adagrad as learning rate strategies. You can find the animation [here](#) or click on the images above.

# Batch Normalization

---

# BATCH NORMALIZATION

- Batch Normalization (BatchNorm) is an extremely popular technique that improves the training speed and stability of deep neural nets.
- It is an extra component that can be placed between each layer of the neural network.
- It works by changing the "distribution" of activations at each hidden layer of the network.
- We know that it is sometimes beneficial to normalize the inputs to a learning algorithm by shifting and scaling all the features so that they have 0 mean and unit variance.
- BatchNorm applies a similar transformation to the activations of the hidden layers (with a couple of additional tricks).

# BATCH NORMALIZATION

- For a hidden layer with neurons  $z_j$ ,  $j = 1, \dots, J$ , BatchNorm is applied to each  $z_j$  by considering the activations of  $z_j$  **over a given minibatch** of inputs.
- Let  $z_j^{(i)}$  denote the activation of  $z_j$  for input  $x^{(i)}$  in the minibatch (of size  $m$ ).
- The mean and variance of the activations are

$$\mu_j = \frac{1}{m} \sum_i^m z_j^{(i)}$$
$$\sigma_j^2 = \frac{1}{m} \sum_i^m (z_j^{(i)} - \mu_j)^2$$

- Each  $z_j^{(i)}$  is then normalized

$$\tilde{z}_j^{(i)} = \frac{z_j^{(i)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

where a small constant,  $\epsilon$ , is added for numerical stability.

# BATCH NORMALIZATION

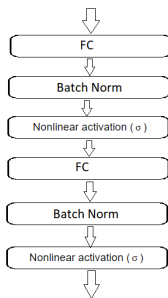
- It may not be desirable to normalize the activations in such a rigid way because potentially useful information can be lost in the process.
- Therefore, we commonly let the training algorithm decide the "right amount" of normalization by allowing it to re-shift and re-scale  $\tilde{z}_j^{(i)}$  to arrive at the batch normalized activation  $\hat{z}_j^{(i)}$ :

$$\hat{z}_j^{(i)} = \gamma_j \tilde{z}_j^{(i)} + \beta_j$$

- $\gamma_j$  and  $\beta_j$  are learnable parameters that are also tweaked by backpropagation.
- $\hat{z}_j^{(i)}$  then becomes the input to the next layer.
- Note: The algorithm is free to scale and shift each  $\tilde{z}_j^{(i)}$  back to its original (unnormalized) value.

# BATCH NORMALIZATION: ILLUSTRATION

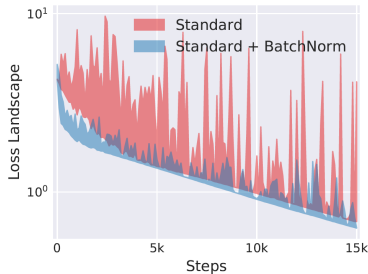
- Recall:  $z_j = \sigma(W_j^T x + b_j)$
- So far, we have applied batch-norm to the activation  $z_j$ . It is possible (and more common) to apply batch norm to  $W_j^T x + b_j$  *before* passing it to the nonlinear activation  $\sigma$ .



**Figure:** FC = Fully Connected layer. BatchNorm is applied *before* the nonlinear activation function.

# BATCH NORMALIZATION

- The key impact of BatchNorm on the training process is this: It reparametrizes the underlying optimization problem to **make its landscape significantly more smooth**.
- One aspect of this is that the loss changes at a smaller rate and the magnitudes of the gradients are also smaller (see Santurkar et al. 2018).



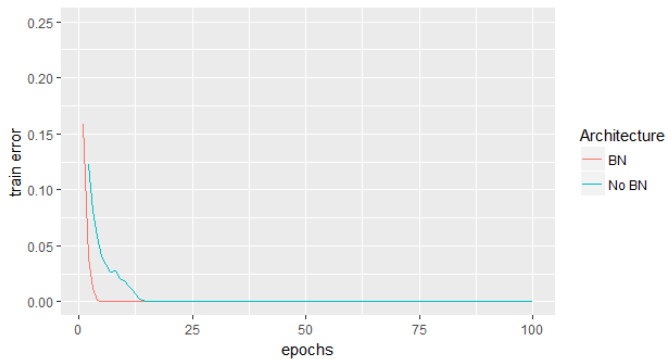
# BATCH NORMALIZATION: PREDICTION

- Once the network has been trained, how can we generate a prediction for a single input (either at test time or in production)?
- One option is to feed the entire training set to the (trained) network and compute the means and standard deviations.
- More commonly, during training, an exponentially weighted running average of each of these statistics over the minibatches is maintained.
- The learned  $\gamma$  and  $\beta$  parameters are then used (in conjunction with the running averages) to generate the output.

# BATCH NORMALIZATION

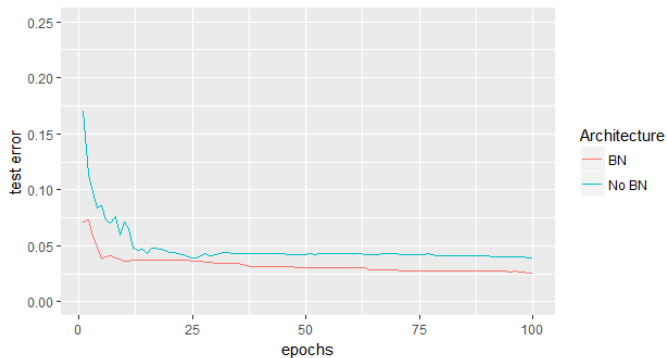
- For our final benchmark in this chapter we compute two models to predict the mnist data.
- One will extend our basic architecture such that we add batch normalization to all hidden layers.
- We use SGD as optimizer with a momentum of 0.9, a learning rate of 0.03 and weight decay of 0.001.

# BATCH NORMALIZATION



Batch Normalization accelerated learning.

# BATCH NORMALIZATION



Batch Normalization resulted in a lower test error.

# REFERENCES



Ian Goodfellow, Yoshua Bengio and Aaron Courville (2016)

Deep Learning

<http://www.deeplearningbook.org/>



Yann Dauphin, Razvan Pascanu, Çağlar Gülçehre, Kyunghyun Cho, Surya Ganguli, Yoshua Bengio (2014)

Identifying and attacking the saddle point problem in high-dimensional non-convex optimization

<https://arxiv.org/abs/1406.2572>



Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, Tom Goldstein (2017)

Visualizing the Loss Landscape of Neural Nets

<https://arxiv.org/abs/1712.09913>

# REFERENCES / 2



Tim Dettmers (2015)

Deep Learning in a Nutshell: History and Training

[https://devblogs.nvidia.com/  
deep-learning-nutshell-history-training/](https://devblogs.nvidia.com/deep-learning-nutshell-history-training/)



Hafidz Zulkifli (2018)

Understanding Learning Rates and How It Improves Performance in Deep Learning

<https://towardsdatascience.com>



Ilya Loshchilov, Frank Hutter (2016)

SGDR: Stochastic Gradient Descent with Warm Restarts

<https://arxiv.org/abs/1608.03983>

# REFERENCES / 3



Jeremy Jordan (2018)

Setting the learning rate of your neural network

[\*https://www.jeremyjordan.me/nn-learning-rate/\*](https://www.jeremyjordan.me/nn-learning-rate/)



Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, Aleksander Madry (2018)

How Does Batch Normalization Help Optimization?

[\*https://arxiv.org/abs/1805.11604\*](https://arxiv.org/abs/1805.11604)



Akshay Chandra (2015)

Learning Parameters, Part 2: Momentum-Based & Nesterov Accelerated Gradient Descent

[\*https://towardsdatascience.com/learning-parameters-part-2-a190bef2d12\*](https://towardsdatascience.com/learning-parameters-part-2-a190bef2d12)

# Hyperparameter Tuning

---

# WHAT IS A HYPERPARAMETER?

- **Tuning** is the process of selecting the best hyperparameters  $\lambda$  for a learner. Hyperparameters are parameters of the learner itself (e.g. learning rate, batch size, number of hidden layers), as opposed to the model parameters  $\theta$  found by training.
- Guitar analogy: hyperparameters are like the tuning pegs; finding the best  $\theta$  (playing the guitar) is a separate process that depends on first being in tune.
- Just as a badly-tuned guitar won't perform well, poorly chosen hyperparameters can drastically hurt model performance. Tuning helps find a balance between under- and overfitting.

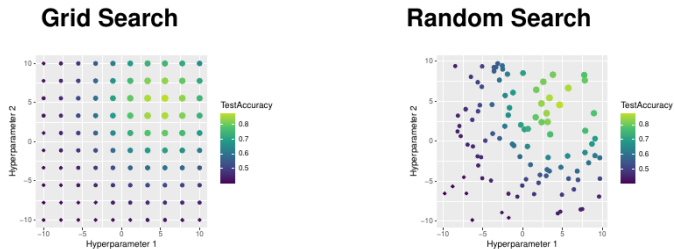
# WHAT IS A HYPERPARAMETER? / 2

- Key training hyperparameters for a neural network include the learning rate (and its schedule), batch size, number of epochs, weight decay / regularization strength, and network width/depth. This session's Advanced Optimization section already covered several of these in depth.

# WHY TUNING IS HARD

- The set of all configurations is the **hyperparameter space**  $\Lambda$ . It grows exponentially: two hyperparameters with 10 possible values each already give  $10 \cdot 10 = 100$  configurations to evaluate.
- Evaluating a configuration  $\lambda \in \Lambda$  is a **black box**: training a network and measuring validation performance is the only way to know how good it is: there is no gradient or closed form to exploit.
- Naive tuners search  $\Lambda$  directly:
  - **Grid search**: evaluate a regular grid of configurations.
  - **Random search**: sample configurations at random. Often more efficient than grid search at the same budget, since it explores more distinct values per hyperparameter.

# WHY TUNING IS HARD / 2



**Figure:** Grid search (left) vs. random search (right) over two hyperparameters.  
Source: SLDS LMU, I2ML.

# GRID SEARCH VS. RANDOM SEARCH

- Both are trivial to implement, support any parameter type, and parallelize trivially.
- **Grid search** discretizes each hyperparameter into a fixed set of values and tries every combination. Downsides: scales combinatorially, wastes evaluations on clearly bad regions, and the choice of grid points is arbitrary.
- **Random search** samples configurations uniformly at random instead. It is an *anytime* algorithm (stop whenever the budget runs out) and never repeats the same value for a given hyperparameter twice.

## GRID SEARCH VS. RANDOM SEARCH / 2

- **Key insight:** if only one of two hyperparameters actually matters, a fixed evaluation budget gives random search far more distinct values of the important one than grid search does. Grid search "wastes" points repeating values of the irrelevant hyperparameter. This is a common situation in practice, e.g. when one hyperparameter barely affects performance.

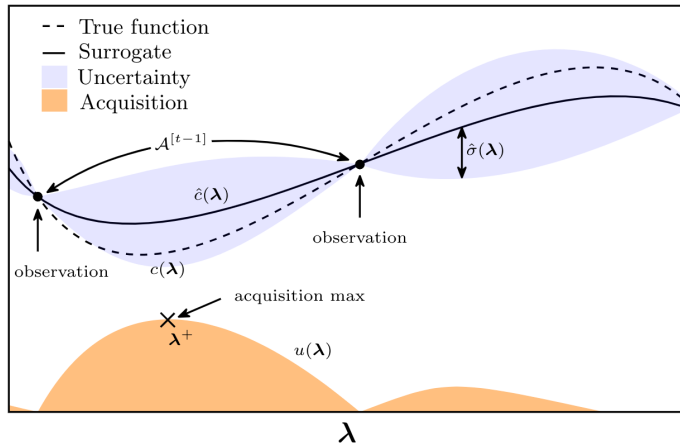
# BAYESIAN OPTIMIZATION

- Grid and random search never use past evaluations to guide future ones. **Bayesian Optimization (BO)** does, by iterating:
  - ➊ Fit a **surrogate model**  $\hat{c}(\lambda)$  (e.g. a Gaussian Process) to the configurations evaluated so far, with an uncertainty estimate  $\hat{\sigma}(\lambda)$ .
  - ➋ Propose the next configuration by maximizing an **acquisition function** that trades off exploring uncertain regions against exploiting regions the surrogate predicts are good (e.g. expected improvement).
  - ➌ Evaluate the proposed configuration, add it to the archive, repeat.

# BAYESIAN OPTIMIZATION / 2

- Because the surrogate guides the search toward promising regions, BO typically needs far fewer evaluations than grid or random search to reach a good configuration, which matters since each evaluation means training a network.

# BAYESIAN OPTIMIZATION / 3



# BAYESIAN OPTIMIZATION / 4

**Figure:** Surrogate model  $\hat{c}(\lambda)$  with uncertainty band (blue) approximating the true cost  $c(\lambda)$  (dashed), and the resulting acquisition function  $u(\lambda)$  (orange) used to pick the next point  $\lambda^+$ . Source: SLDS LMU, I2ML.

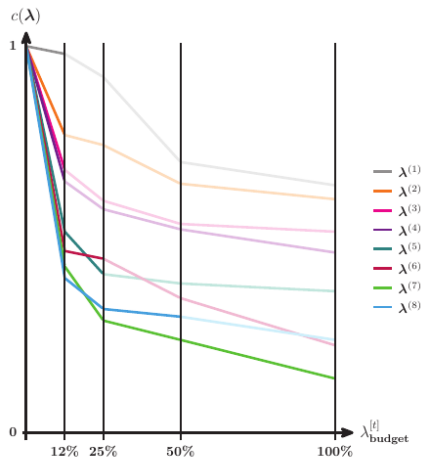
# MULTI-FIDELITY OPTIMIZATION

- Training a network to full convergence just to evaluate one configuration is expensive. **Multi-fidelity** methods instead train with a cheap approximation first (e.g. fewer epochs or a data subset; the **fidelity**) and only spend the full budget on promising candidates.
- **Successive Halving**: train all candidate configurations with a small fraction of the full budget, discard the worse half, double the budget for survivors, and repeat until one configuration (or the full budget) remains.

# MULTI-FIDELITY OPTIMIZATION / 2

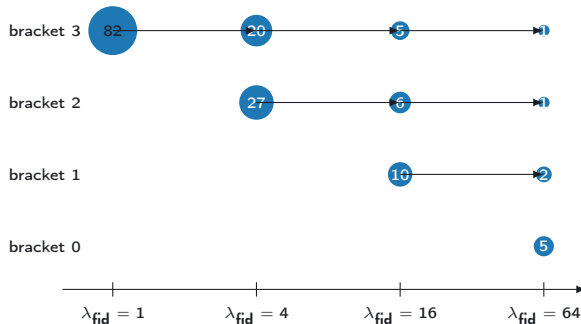
- **Hyperband** fixes Successive Halving's main weakness: a genuinely good configuration can still be killed off too early depending on the starting budget. It does this by repeating Successive Halving with several different starting budgets ("brackets") and aggregating.
- Directly relevant to DL: early-stopping bad hyperparameter configurations after a few epochs (rather than training every candidate to convergence) is exactly this idea.

# MULTI-FIDELITY OPTIMIZATION / 3



# HYPERBAND

Several Successive-Halving brackets at once



# HYPERBAND / 2

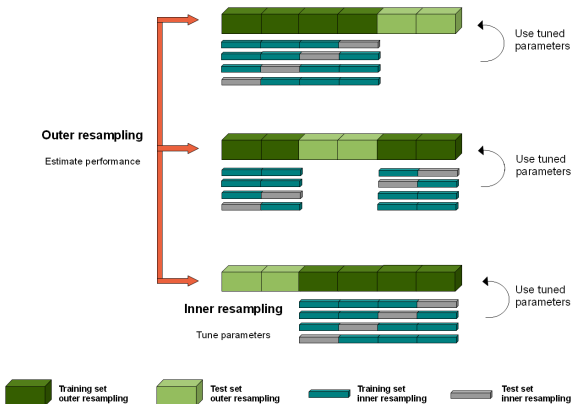
**Figure:** Hyperband for  $\eta = 4$ : each row is one Successive-Halving “bracket”, started with a different (starting budget, number of configurations) trade-off — bracket 3 starts cheap with many configurations, bracket 0 spends the full budget on few. Circle labels are surviving configuration counts at each stage; circle size is illustrative, not to scale. Numbers verified from SLDS LMU, I2ML, “Hyperparameter Tuning: Advanced Tuning Techniques” (see References).

# THE UNTOUCHED-TEST-SET PRINCIPLE

- Once tuning has found a configuration  $\lambda^*$ , how well does it actually perform? We cannot reuse the data used for tuning to also estimate performance, since that would give an optimistically biased estimate.
- To get an unbiased estimate, an **untouched test set** is needed: the data used to select  $\lambda^*$  (train/validation split) must be kept separate from the test set used only once, at the very end, to report performance.
- **Nested resampling** generalizes this to reduce the variance of the performance estimate: an outer resampling loop estimates generalization performance, while an inner resampling loop (on only the outer-training data) is used to tune  $\lambda$ .

# THE UNTOUCHED-TEST-SET PRINCIPLE

/ 2



# THE UNTOUCHED-TEST-SET PRINCIPLE

/ 3

---

**Figure:** Nested resampling: an inner loop tunes hyperparameters, an outer loop estimates performance of the tuned model. Source: SLDS LMU, I2ML.

# REFERENCES



SLDS LMU. *Introduction to Machine Learning (I2ML)*, chapter “Tuning & Nested Resampling: In a Nutshell”. [https://slds-lmu.github.io/i2ml/chapters/09\\_tuning/09-00-nutshell-tuning/](https://slds-lmu.github.io/i2ml/chapters/09_tuning/09-00-nutshell-tuning/)



SLDS LMU. *Introduction to Machine Learning (I2ML)*, chapter “Hyperparameter Tuning: Basic Techniques”. [https://slds-lmu.github.io/i2ml/chapters/09\\_tuning/09-03-basicalgos/](https://slds-lmu.github.io/i2ml/chapters/09_tuning/09-03-basicalgos/)



SLDS LMU. *Introduction to Machine Learning (I2ML)*, chapter “Hyperparameter Tuning: Advanced Tuning Techniques”. [https://slds-lmu.github.io/i2ml/chapters/09\\_tuning/09-04-tuning-advanced/](https://slds-lmu.github.io/i2ml/chapters/09_tuning/09-04-tuning-advanced/)

# Neural Architecture Search

---

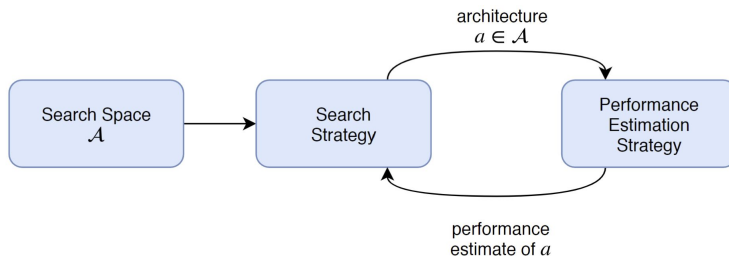
# NEURAL ARCHITECTURE SEARCH:

## MOTIVATION

- Manually designing state-of-the-art architectures (ResNet, Inception, ...) is time-consuming: complex architectures are the result of years of trial and error by experts.
- **Neural Architecture Search (NAS)** automates this: the goal is to automatically find neural architectures with strong performance, optionally subject to a resource constraint.
- A decade-old idea, but mainstream only since 2017; initially extremely expensive (hundreds of GPUs for weeks), but by now several methods promise low overhead over training a single model.

# NAS COMPONENTS AND DEFINITION

- NAS methods are described by three components: a **search space**  $\mathcal{A}$  of candidate architectures, a **search strategy** that picks which architecture  $a \in \mathcal{A}$  to try next, and a **performance estimation strategy** that estimates how good  $a$  is.



# NAS COMPONENTS AND DEFINITION / 2

**Figure:** The NAS loop: search strategy proposes an architecture, performance estimation strategy evaluates it. Source: Lindauer & Eggenberger, AutoML.org (ESSAI 2023).

- Formally: given a search space  $\mathcal{A}$ , dataset  $D$ , training pipeline  $P$ , and a time/compute budget  $t$ , find  $a \in \mathcal{A}$  within budget  $t$  with the highest validation accuracy when trained on  $D$  with  $P$ :

$$\min_{a \in \mathcal{A}} \mathcal{L}_{\text{val}}(w^*(a), a) \quad \text{s.t.} \quad w^*(a) \in \arg \min_w \mathcal{L}_{\text{train}}(w, a).$$

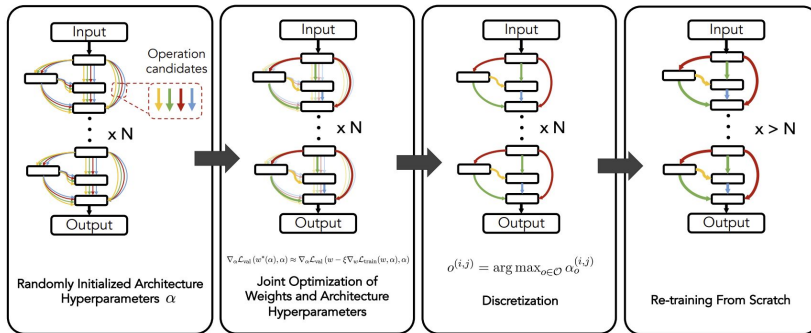
# ONE EXAMPLE: DARTS

## (DIFFERENTIABLE ARCHITECTURE SEARCH)

---

- Search strategies range from evolutionary algorithms and reinforcement learning to Bayesian optimization, all expensive, since naively each candidate architecture needs to be trained to be evaluated.
- **DARTS** (Liu et al. 2018) is one way to make this affordable: it relaxes the discrete choice of operation on each edge into a continuous, differentiable weight, so the architecture itself can be optimized with gradient descent instead of a separate search loop, with no RL or evolution needed.

# ONE EXAMPLE: DARTS (DIFFERENTIABLE ARCHITECTURE SEARCH) / 2



# ONE EXAMPLE: DARTS

## (DIFFERENTIABLE ARCHITECTURE SEARCH) / 3

**Figure:** DARTS pipeline: (1) randomly initialize architecture weights  $\alpha$ , (2) jointly optimize  $\alpha$  and network weights  $w$ , (3) discretize by taking the arg max operation per edge, (4) retrain the resulting fixed architecture from scratch.  
Source: Lindauer & Eggenberger, AutoML.org (ESSAI 2023).

- This “one-shot, differentiable” trick is one instance of a broader NAS toolbox (ENAS, evolutionary search, Bayesian optimization, zero-cost proxies, ...): the point isn’t the specific algorithm, but that architecture design itself can be automated and made tractable.

# NAS: FINAL THOUGHTS

- Good architectures via NAS are feasible within a few hours today, down from hundreds of GPU-hours for the earliest methods.
- Gradient-based methods (DARTS and successors) are fastest but least stable; Bayesian optimization with the right tricks remains highly competitive.
- NAS is still limited in what it can discover: it is unclear whether NAS could have found genuinely novel architectures like the Transformer, and NAS is not easily applicable to very expensive models (e.g. LLMs), where even one full training run is already costly.

# REFERENCES



Lindauer, M., & Eggenberger, K. (2023). *NAS: Neural Architecture Search*. ESSAI SummerSchool 2023, AutoML.org. [https://www.automl.org/wp-content/uploads/2023/07/3\\_ESSAI\\_NAS-1.pdf](https://www.automl.org/wp-content/uploads/2023/07/3_ESSAI_NAS-1.pdf)

# LICENSE & ATTRIBUTION

- SGD Recap, Momentum, Learning Rates, Adaptive Optimizers, and Batch Normalization adapted from **Introduction to Deep Learning (I2DL)**, Prof. Dr. Bernd Bischl, SLDS, LMU Munich.  
[https://github.com/slds-lmu/lecture\\_i2dl](https://github.com/slds-lmu/lecture_i2dl) — licensed **CC BY 4.0**:  
<https://creativecommons.org/licenses/by/4.0/>
- Hyperparameter Tuning assembled from **Introduction to Machine Learning (I2ML)**, SLDS, LMU Munich.  
<https://github.com/slds-lmu/i2ml> — licensed **MIT**.

## LICENSE & ATTRIBUTION / 2

- Neural Architecture Search adapted (and substantially condensed) from Lindauer & Eggenberger, *NAS: Neural Architecture Search*, ESSAI SummerSchool 2023, AutoML.org — licensed **CC BY-SA**. This section of the deck, including its two figures, is shared under the same Attribution-ShareAlike terms: <https://creativecommons.org/licenses/by-sa/4.0/>
- Content in this deck has been merged, reordered, and (for NAS) substantially condensed from the originals; not used verbatim.